

Linear Queue

Source Code

```
#include <iostream>
using namespace std;

class Queue{
public:
    int* arr;
    int FRONT;
    int REAR;
    int MAX;
    Queue(int MAX_){
        arr = new int[MAX_];
        FRONT = -1;
        REAR = -1;
        MAX = MAX_;
    }
    int Enqueue(int ITEM){
        if (REAR == MAX -1) {
            cout << "Queue Overflow!!" << endl;
            return 1;
        }
        if (FRONT == -1){
            FRONT = 0;
            REAR = 0;
        }
        else REAR += 1;
        arr[REAR] = ITEM;
        return 0;
    }
    int Dequeue(){
        if (FRONT == -1 || FRONT > REAR){
            cout << "Queue Underflow!!" << endl;
            return 1;
        }
        return arr[FRONT];
        if (FRONT == REAR){
            FRONT = -1;
            REAR = -1;
        }
        else FRONT += 1;
        return 0;
    }
    void showQueue(){
        cout << "Printing Queue Status: " << endl;
        for (int i = 0; i < MAX; i++){
            cout << arr[i] << ", ";
        }
        cout << endl;
    }
};
```

```

    }
};

int main(){
    Queue myQueue(5);
    cout << "Attempting to Dequeue empty Queue!" << endl;
    myQueue.Dequeue();
    int item;
    cout << "Enter a number to enqueue. "; cin >> item;
    myQueue.Enqueue(item);
    myQueue.showQueue();
    cout << "Now Dequeueing this number : " << myQueue.Dequeue() << endl;
    return 0;
}

```

Output:

```

pawanadhikari@Pawans-MacBook-Air 03_Queues % ./01_LinQueue
Attempting to Dequeue empty Queue!
Queue Underflow!!
Enter a number to enqueue. 50
Printing Queue Status:
50, 0, 0, 0, 0,
Now Dequeueing this number : 50

```

Discussions

- This program demonstrates a basic linear queue with FIFO behavior using **FRONT** and **REAR** indices.
- It shows the normal queue conditions clearly, including underflow when dequeueing from an empty queue and overflow when the array becomes full.
- The queue status output helps visualize how the array stores items after enqueue and dequeue operations.

Circular Queue

Source Code

```

#include <iostream>
using namespace std;

class CircQueue{
public:
    int* arr;
    int FRONT;
    int REAR;
    int MAX;
    CircQueue(int MAX_){
        arr = new int[MAX_];
    }
}

```

```
        FRONT = -1;
        REAR = -1;
        MAX = MAX_;
    }
    int Enqueue(int ITEM){
        if ((REAR + 1)%MAX == FRONT) {
            cout << "Queue Overflow!!" << endl;
            return 1;
        }
        if (FRONT == -1){
            FRONT = 0;
            REAR = 0;
        }
        else REAR = (REAR+1)%MAX;
        arr[REAR] = ITEM;
        return 0;
    }
    int Dequeue(){
        if (FRONT == -1){
            cout << "Queue Underflow!!" << endl;
            return 1;
        }
        int ITEM = arr[FRONT];
        if (FRONT == REAR){
            FRONT = -1;
            REAR = -1;
        }
        else FRONT = (FRONT+1)%MAX;
        return ITEM;
    }
    void showQueue(){
        cout << "Printing Queue Status: " << endl;
        for (int i = 0; i < MAX; i++){
            cout << arr[i] << ", ";
        }
        cout << endl;
    }
};

int main(){
    int max;
    cout << "Enter Queue Size: "; cin>>max;
    CircQueue myQueue(max);

    cout << "Attempting to Dequeue empty Queue." << endl;
    myQueue.Dequeue();

    int item, n;
    cout << "Enter a number to enqueue. "; cin >> item;
    cout << "Enter number of times to enqueue this: "; cin >> n;
    for (int i=0; i<n; i++) myQueue.Enqueue(item);

    myQueue.showQueue();
}
```

```

    cout << "Now Dequeueing; Enter number of times to dequeue: "; cin >> n;

    for (int i=0; i<n; i++){
        cout << "Dequeued: " << myQueue.Dequeue() << endl;
    }

    myQueue.showQueue();
    return 0;
}

```

Output:

```

pawanadhikari@Pawans-MacBook-Air 03_Queues % gcr 02_CircQueue.cpp
Enter Queue Size: 10
Attempting to Dequeue empty Queue.
Queue Underflow!!
Enter a number to enqueue. 5
Enter number of times to enqueue this: 7
Printing Queue Status:
5, 5, 5, 5, 5, 5, 5, 0, 0, 0,
Now Dequeueing; Enter number of times to dequeue: 6
Dequeued: 5
Dequeued: 5
Dequeued: 5
Dequeued: 5
Dequeued: 5
Dequeued: 5
Printing Queue Status:
0, 0, 0, 0, 0, 0, 5, 0, 0, 0,

```

Discussions

- This program implements a circular queue, which reuses freed space by moving **FRONT** and **REAR** with modulo arithmetic.
- It avoids the wasted space problem of a simple linear queue and allows efficient enqueue and dequeue operations.
- The example output shows wrap-around behavior and also demonstrates both overflow and underflow handling.

Double Ended Queue

Source Code

```

#include <iostream>
using namespace std;

class CircDeque{
public:

```

```
int* arr;
int FRONT;
int REAR;
int MAX;
CircDeque(int MAX_){
    arr = new int[MAX_];
    FRONT = -1;
    REAR = -1;
    MAX = MAX_;
}
int EnqueueRear(int ITEM){
    if ((FRONT==0 && REAR==MAX-1) || (FRONT==REAR+1)) {
        cout << "Queue Overflow!!" << endl;
        return 1;
    }
    if (FRONT == -1){
        FRONT = 0;
        REAR = 0;
    }
    else if (REAR==MAX-1) REAR=0;
    else REAR += 1;
    arr[REAR] = ITEM;
    return 0;
}
int DequeueFront(){
    if (FRONT == -1){
        cout << "Queue Underflow!!" << endl;
        return 1;
    }
    int ITEM = arr[FRONT];
    arr[FRONT]=0;
    if (FRONT == REAR){
        FRONT = -1;
        REAR = -1;
    }
    else if (FRONT==MAX-1) FRONT=0;
    else FRONT += 1;
    return ITEM;
}
int EnqueueFront(int ITEM){
    if ((FRONT==0 && REAR == MAX -1) || (FRONT>REAR)){
        cout << "Queue Overflow!!" << endl;
        return 1;
    }
    if (FRONT == -1){
        FRONT = 0;
        REAR = 0;
    }
    else if (FRONT==0) FRONT = MAX - 1;
    else FRONT -= 1;
    arr[FRONT] = ITEM;
    return 0;
}
int DequeueRear(){
```

```
        if (FRONT == -1){
            cout << "Queue Underflow!!" << endl;
            return 1;
        }
        int ITEM = arr[REAR];
        arr[REAR]=0;
        if (FRONT == REAR){
            FRONT = -1;
            REAR = -1;
        }
        else if (REAR == 0) REAR = MAX - 1;
        else REAR = REAR - 1;
        return ITEM;
    }
    void showQueue(){
        cout << "Printing Queue Status: " << endl;
        for (int i = 0; i < MAX; i++){
            cout << arr[i] << ", ";
        }
        cout << endl;
        cout << "Front at " << FRONT << "\t REAR at " << REAR << endl;
    }
};

int main(){
    int max;
    cout << "Enter Queue Size: "; cin>>max;
    CircDeque myQueue(max);

    cout << "Attempting to Dequeue empty deque from front." << endl;
    myQueue.DequeueFront();

    int item, n;
    cout << "Enter a number to enqueue from rear: "; cin >> item;
    cout << "Enter number of times to enqueue this: "; cin >> n;
    for (int i=0; i<n; i++) myQueue.EnqueueRear(item);

    myQueue.showQueue();

    cout << "Now Dequeueing from front; Enter number of times to dequeue: "; cin >> n;

    for (int i=0; i<n; i++){
        cout << "Dequeued: " << myQueue.DequeueFront() << endl;
    }

    myQueue.showQueue();

    cout << "Enter a number to enqueue from front: "; cin >> item;
    cout << "Enter number of times to enqueue this: "; cin >> n;
    for (int i=0; i<n; i++) myQueue.EnqueueFront(item);

    myQueue.showQueue();
```

```

    cout << "Now Dequeueing from rear; Enter number of times to dequeue: ";
    cin >> n;

    for (int i=0; i<n; i++){
        cout << "Dequeued: " << myQueue.DequeueRear() << endl;
    }

    myQueue.showQueue();

    return 0;
}

```

Output:

```

pawanadhikari@Pawans-MacBook-Air 03_Queue % ./03_Deque
Enter Queue Size: 15
Attempting to Dequeue empty deque from front.
Queue Underflow!!
Enter a number to enqueue from rear: 50
Enter number of times to enqueue this: 20
Queue Overflow!!
Queue Overflow!!
Queue Overflow!!
Queue Overflow!!
Queue Overflow!!
Printing Queue Status:
50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50,
Front at 0      REAR at 14
Now Dequeueing from front; Enter number of times to dequeue: 5
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50
Printing Queue Status:
0, 0, 0, 0, 0, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50,
Front at 5      REAR at 14
Enter a number to enqueue from front: 4
Enter number of times to enqueue this: 6
Queue Overflow!!
Printing Queue Status:
4, 4, 4, 4, 4, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50,
Front at 0      REAR at 14
Now Dequeueing from rear; Enter number of times to dequeue: 20
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 50

```

```
Dequeued: 50
Dequeued: 50
Dequeued: 50
Dequeued: 4
Dequeued: 4
Dequeued: 4
Dequeued: 4
Dequeued: 4
Dequeued: Queue Underflow!!
1
Dequeued: Queue Underflow!!
1
Dequeued: Queue Underflow!!
1
Dequeued: Queue Underflow!!
1
Dequeued: Queue Underflow!!
1
Printing Queue Status:
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
Front at -1      REAR at -1
```

Discussions

- This program implements a double ended queue, or deque, where insertion and deletion can happen at both the front and the rear.
- It uses circular movement of **FRONT** and **REAR** to support efficient operations on both ends without shifting elements.
- The output shows how the deque handles overflow, underflow, and wrap-around while operating from both sides.